LBRO Project Q&A

Detailed questions and answers about the LBRO platform: architecture, security, ingestion, incidents, evidence, machine learning, frontend, testing and open items. Status reflects the audit of 4 November 2026.

1. Project overview

Q1. What is LBRO?

LBRO (Law-aware Breach Response Orchestrator) is a security incident response platform. It ingests security events from applications and agents, classifies them with a machine-learning model, opens and tracks incidents, stores evidence with a chain of custody, and tracks legal notification deadlines under frameworks such as GDPR, HIPAA and DPDPA. It is built as an academic and portfolio project, and its target users are SOC analysts, security administrators, and integrators who send events using project API keys.

Q2. Who are the intended users?

There are three main groups. SOC analysts triage incidents and review ML verdicts. Security administrators manage projects, users, and evidence policy. Integrators are developers whose applications send events to LBRO through the Bearer proj_* ingest key. Access is controlled by role-based access control, described in section 3.

Q3. What is the current release status?

The project is a production-hardened demo. The most recent readiness verdict was READY WITH CONDITIONS. Three conditions remain open: the full Docker Compose lifecycle smoke test has not been completed, the Playwright integration tests and the 20-run login performance test have not been started, and the ML evidence needed a clear labeling decision before it could be presented as production-grade. Nothing has been pushed to GitHub yet.

Q4. What does the repository contain?

The repository has a FastAPI backend under backend/app, a React 18 and Vite single-page frontend under frontend/src, Alembic migrations 001 through 018, SQS-driven workers, Docker and Docker Compose files, Terraform for AWS under terraform/, client SDKs for Go, Java, Node.js and Python under sdk/, a log-shipping agent under agent/, documentation in docs/, and a large set of audit and certification reports at the repository root.

Q5. Why is LBRO described as law-aware?

Each incident can carry data-classification flags such as personal data, health data, and whether GDPR, HIPAA or DPDPA notification applies. The platform uses these flags to compute notification deadlines and to show a compliance view. The compliance rules themselves are still marked as unreviewed by a lawyer, so the feature should be presented as a workflow aid rather than legal advice.

2. Architecture and runtime

Q6. How do the components talk to each other?

A browser reaches an nginx container on port 80. Requests under /api/ are proxied to the FastAPI application on port 8000, which uses PostgreSQL 16 through asyncpg and SQLAlchemy. Asynchronous work such as ML classification and notification retries is placed on Amazon SQS queues and consumed by a separate worker process started with python -m app.workers.main. External applications send events straight to the API using a Bearer proj_* key.

Q7. Which Python and framework versions are pinned?

The backend targets Python 3.12. The pinned runtime includes FastAPI 0.141.1, Uvicorn 0.32.1, SQLAlchemy 2.0.36 with asyncpg 0.30.0, Alembic 1.14.0, Pydantic 2.10.3, scikit-learn 1.7.2, numpy 1.26.4, joblib 1.4.2, and reportlab 4.2.5 for PDF reports. The frontend uses React 18, Vite 6, TypeScript 5.3, Tailwind, TanStack Query, and Zustand. Numpy 1.26.4 has no Python 3.13 wheel for Windows, so developers must use Python 3.12.

Q8. How are the database migrations managed?

Schema changes live in backend/app/migrations/versions as Alembic revisions numbered 001 to 018. The current head is 018_integration_context, and the audit confirmed there is a single head. Migrations run with python scripts/run_migrations.py or with alembic upgrade head. In Docker Compose a dedicated migrate service runs them before the API starts.

Q9. Which parts of the system are single-instance only?

The server-sent event stream used for live updates and the rate limiter both keep their state in process memory. They behave correctly only when the API runs as a single replica. Redis is referenced in the configuration as REDIS_URL but is not used, so scaling out would need a shared store for these two features first.

Q10. What is the difference between the liveness and readiness endpoints?

GET /health is a liveness check that only confirms the process is running. GET /health/ready is the readiness endpoint, implemented in app/main.py, and it closes audit item AUD-001. The owner has an open decision on whether readiness should also check the database, SQS and the ML bundle.

Q11. How is the Compose stack organised?

The development compose file defines api, frontend, localstack, migrate, postgres, seed and worker services. The production compose file defines api, frontend, postgres and worker. The audit confirmed that both files parse and that docker compose config validates them, but a full lifecycle smoke test has not run because the sandbox could not pull base images from Docker Hub.

3. Authentication, authorization and security

Q12. How do users log in?

Humans log in with an email address and a password. A successful login returns a JWT signed with HS256 and valid for 30 minutes, as set in config.py. Logout revokes the token's jti by storing it in the revoked_tokens table, and the revocation is enforced on the /me endpoint, which is covered by an integration test.

Q13. How is brute-force login handled?

The test suite includes test_account_lockout_after_five_failures, which confirms that an account is locked after five consecutive failed attempts. The login path also has a timing test, test_login_timing.py, which is intended to check that response timing does not reveal whether an account exists.

Q14. How do integrations authenticate?

Event ingestion uses only Bearer proj_* keys. Each project can have its own keys, and the stored value is a hash, which is verified by a round-trip test in test_api_key_security.py. The legacy X-API-Key header still exists on the human-facing API and is a candidate for removal.

Q15. What roles exist and what can they do?

There is a platform-wide super_admin role and three project-level roles: admin, analyst and viewer. Permissions are defined in app/core/rbac.py. Admins manage the project, analysts work incidents and review ML results, and viewers get read-only access. Every denied request is written to the audit log. The test_rbac.py and test_project_isolation.py suites check both the permission matrix and that one project cannot read another project's data.

Q16. How is rate limiting configured?

The default limit is 60 requests per minute per IP address, set in config.py, and authentication paths are stricter than that. Rate limiting is turned off when ENVIRONMENT is test. The limiter is held in memory, so it only works

on a single replica.

Q17. Which security headers are sent?

Security headers come from both middleware in the API and nginx. They include a Content-Security-Policy and HTTP Strict Transport Security. The current CSP still allows unsafe-inline scripts, so tightening it is a known follow-up.

Q18. What does the security review of the codebase show?

The last scans were gitleaks with zero leaks across 68 commits, bandit with zero High findings, and pip-audit on the pinned runtime requirements with no known vulnerabilities. After this audit, the bandit CI gate passes with zero findings because the justified uses were annotated with nosec comments and one unneeded development host entry was removed. The frontend production dependency audit now reports zero findings after a lockfile update. Development-only npm advisories remain and are not part of the shipped application.

Q19. Is multi-factor authentication available?

Not yet. The user table has schema fields for MFA, but there is no TOTP flow. The owner has an open decision on whether to build TOTP or remove MFA from the security score. Until that decision is made, MFA should not be claimed as a feature.

Q20. What happens when a seeded account is used?

The seed script creates demonstration accounts with documented default passwords. These must be changed before any deployment that is reachable from a network. The README's known limitations section says this explicitly.

4. Ingestion and events

Q21. How does an application send an event?

The application sends an HTTP POST to /api/v1/events with a Bearer proj_* key. The body includes an event_type, a severity (critical, high, medium, low or info), optional source IP, host and application fields, a message, an event timestamp, and a payload of key-value pairs. The endpoint also accepts batches of up to 1000 events in a single request.

Q22. What happens to an event after it arrives?

The event is stored as a SecurityEvent with processing status pending. The backend then runs inline ML classification if the payload has network-flow features, or falls back to a heuristic mapping from event type to attack category. The event is marked processed, and any critical or high event automatically creates an incident. A critical event is downgraded to high when the ML result is only heuristic or needs review.

Q23. How are event types mapped to attack categories?

When no ML features are present, the backend maps event types to CICIDS-style categories. For example, sql_injection maps to Web Attack - Sql Injection, xss to Web Attack - XSS, brute_force and auth_failure to Web Attack - Brute Force, port_scan and firewall_event to PortScan, and windows_event and linux_audit to Infiltration. The custom type maps to BENIGN.

Q24. What is the lbro agent?

The agent in agent/lbro_agent.py is a log shipper that reads local log sources and forwards them to the events endpoint. It identifies itself with a source_agent_version field, which lets the backend record which agent version produced each event.

Q25. What do the official SDKs cover?

The sdk directory contains client libraries for Go, Java, Node.js and Python. Each wraps the events endpoint so that integrators do not need to build the request and authentication headers by hand.

5. Incidents and investigation

Q26. What is stored on an incident?

An incident holds a title, description, severity, status, attack category, ML confidence score, the ML model version, source and destination IP and port, protocol, detection and update times, an analyst-review flag, and a set of data-classification flags. Actions taken on an incident are written as IncidentAction records, and ML outcomes are recorded as automated actions with the model version and coverage.

Q27. How does an incident move through its lifecycle?

Incidents start with status new. Analysts move them through investigation states until they are closed, and the closed-at timestamp is recorded when that happens. Each transition is written to the audit trail with the actor and time, so the history of an incident can be reconstructed.

Q28. How does the ML verdict relate to an incident's severity?

The ML classifier sets the attack category and confidence, and the incident worker only raises or lowers severity when the model is confident and the result is not flagged for analyst review. The title and the original severity come from the sender's message and are stored as submitted. Because of this, an incident can show a high severity from the sender and a BENIGN verdict from the model at the same time.

Q29. What did we change about conflicting verdicts?

We added a banner on the incident overview. It appears when the reported severity is high or critical but the model classifies the event as BENIGN. The banner shows the reported severity, the model's confidence as a percentage, and a reminder that the title and severity came from the sender. The Attack Explanation panel also no longer describes a source IP, port or duration for BENIGN verdicts, because those describe an attack that did not happen.

Q30. Where do the explanations on the incident page come from?

The explanation panels are generated by app/services/incident_explainer.py. It has a fixed entry for each attack category, including plain-English summary text, business and technical impact, likelihood, OWASP mapping, MITRE ATT&CK techniques and recommended fixes. The panel adds context about the source IP, the targeted port and the flow duration only when the category is an attack. These explanations are static templates, not generated by a language model.

Q31. What is the Threat Detection page showing?

The ML Analysis page shows predictions made today, average confidence, the number of items needing review, the active model version and status, and an attack distribution. The model accuracy and F1 cards are read from the signed bundle's held-out evaluation. Before the fix described below, those cards showed N/A even for the real model.

Q32. What was wrong with the Threat Detection metrics, and how was it fixed?

The signed CICIDS2017 manifest stores its evaluation under held_out_test, but the API only read a block named realistic. The API therefore saw no metrics and displayed N/A. The fix maps held_out_test into the realistic block when the bundle is not demo-synthetic, so the page now shows accuracy 0.9983 and macro F1 0.8773 from the held-out split. The manifest itself was not edited, which keeps its HMAC signature valid. The feature and class count defaults were also changed from hardcoded 78 and 15 to the real 77 and 15.

6. Evidence and chain of custody

Q33. How is evidence stored?

Evidence files are stored in PostgreSQL in a file_data column, along with their filename, content type, size and a SHA-256 hash. The model also has legacy S3 fields, and the README and the implementation disagree on whether WORM storage in S3 is the intended long-term design. That decision is still open.

Q34. Which evidence is captured automatically?

When an event creates an incident, the ingestion path stores the raw event as a network-flow JSON file and an ingestion-event JSON file. These are marked immutable, and each one gets a chain-of-custody entry with the

action ingested and a note that it was auto-captured from the POST to /api/v1/events.

Q35. How does hash verification work?

The verify endpoint at POST /api/v1/evidence/{id}/verify recomputes the SHA-256 hash over the stored file_data and compares it with the hash recorded at upload or ingestion. The response contains ok, verified, hash_matched and the computed hash. The verified and hash_matched fields were added so the frontend can show the result. Before that change, the page showed every item as a Mismatch even when the hashes matched.

Q36. What does a Mismatch badge mean?

It means the recomputed hash does not equal the stored hash. After the field fix, the badge should only appear when the stored bytes have actually changed. The sandbox test confirmed the verify path works, but the badge should be checked against a live record on a running backend before it is used in a demonstration.

Q37. Who can download or verify evidence?

Downloading and verifying evidence requires the DOWNLOAD_EVIDENCE permission. Actions on evidence are written to the audit log together with the actor and the client IP address.

7. Machine learning

Q38. What does the ML classifier predict?

The classifier is a multi-class model trained on the CICIDS2017 dataset. It predicts one of 15 classes: BENIGN, Bot, DDoS, DoS GoldenEye, DoS Hulk, DoS Slowhttptest, DoS slowloris, FTP-Patator, Heartbleed, Infiltration, PortScan, SSH-Patator, Web Attack - Brute Force, Web Attack - Sql Injection, and Web Attack - XSS. Its input is a set of 77 flow features in a fixed canonical order defined in app/ml/features.py.

Q39. Which algorithm was chosen and why?

The shipped model is a HistGradientBoostingClassifier with max_iter 100, class_weight balanced, and random state 42. It was chosen over Naive Bayes, Random Forest and a Decision Tree. On a 119,616-flow sample, HistGradientBoosting scored macro F1 0.913 on a single split and 0.908 under cross-validation. The Decision Tree scored 0.916 and 0.898, Random Forest 0.885 and 0.890, and Naive Bayes 0.440. The comparison slide in the deck reports these figures.

Q40. Is the model selection fair?

Not entirely, and it should be described that way. The original train_models.py script selected the winner using test-set metrics, which biases the selection. The later trial scripts selected on a validation split, but the comparison numbers above should be presented as informative rather than as a clean benchmark.

Q41. How was the shipped model trained?

It was trained on all 2,830,743 flows from the eight CICIDS2017 MachineLearningCVE daily files, using an 80/20 stratified split with random state 42. Label cleanup maps the mojibake Web Attack labels to the three canonical web attack names, and seven column names are aligned through an alias table. Median imputation values come from the training split only.

Q42. What are the held-out and cross-validation results?

On the 20 percent held-out test set, the model reaches accuracy 0.9983 and macro F1 0.8773. Five-fold stratified cross-validation gives accuracy 0.9975 and macro F1 0.8485 with a standard deviation of 0.021. The cross-validation was run on scikit-learn 1.9.1, which is not the pinned version, so the shipped model's numbers should be read as produced by the training environment that was recorded in the manifest.

Q43. Which attack classes perform poorly?

Rare or hard classes drive most of the error. Heartbleed has only 11 flows in total, and SQL injection has 21 and Infiltration 36. On the held-out set, Web Attack Sql Injection scores F1 0.444, Web Attack XSS 0.504, Web Attack Brute Force 0.717, Bot 0.774 and Infiltration 0.778. Web attacks are hard to separate using flow-level

features alone because they look like ordinary HTTP traffic.

Q44. What is the review threshold?

A prediction that falls below a 75 percent confidence threshold is flagged as needing analyst review. The model's out-of-distribution bounds, which are stored in the preprocessing file, can also trigger a review flag when a feature value falls outside the range seen in training.

Q45. What happens when there are not enough features?

If an event does not carry enough network-flow features, the classifier uses a heuristic fallback instead of the model. The result is labelled with the source heuristic or insufficient_features, and it is always flagged for review. Auto-containment is not triggered on heuristic results.

Q46. How is the model bundle protected?

The bundle directory contains model.joblib, preprocessing.json, classes.json, manifest.json and manifest.sig. The manifest records the SHA-256 of each artifact, the training data hashes, the library versions and the metrics. The manifest is signed with HMAC-SHA256 using the ML_MODEL_SIGNING_KEY. The loader checks the hashes and the signature before it deserializes the joblib file, and it refuses to load an unsigned bundle in production.

Q47. Why does loading check the scikit-learn version?

Pickled scikit-learn models are not guaranteed to load across minor versions. The loader compares the major and minor version recorded in the manifest with the runtime version and rejects a mismatch with the reason code sklearn_version_mismatch. The shipped bundle was trained on 1.7.2, which matches the pinned runtime.

Q48. Is the signature check complete for the shipped model?

The HMAC was calculated with the real signing key from the root environment file, and the file hashes were checked against the manifest. A full load through the application's verifier with the real key has not been run in the audit sandbox, because that would have required copying the signing key there. It should be run once on the developer machine using scripts/verify_model.py with --load-model.

Q49. Can the model be retrained?

Yes. scripts/train_models.py and scripts/train_models_cli.py contain the training pipeline, and app/ml/training/pipeline.py writes the signed manifest after training. Training on the full CICIDS2017 data took several hours, so a subsample was used to test feasibility before the full run.

8. Frontend

Q50. What are the main screens?

The sidebar links to Dashboard, Projects, Incidents, Live Events, Evidence, ML Analysis, the Attack library, Compliance, the Data Map, Reports, Integrations, Settings, Notifications and Profile. The incident detail page has tabs for Overview, Timeline, Raw Event, Evidence, Notes, IOC, Exposure and Obligations, and Related incidents.

Q51. How does the frontend talk to the backend?

The React application uses Axios and TanStack Query for requests, and Zustand for client state. During development, Vite proxies /api requests to port 8000. In production, nginx serves the built single-page app and proxies /api to the backend.

Q52. How is the frontend checked?

The frontend has a TypeScript type-check, an ESLint run with zero warnings allowed, 75 Vitest tests across 12 files using React Testing Library and MSW, and a production build. All four pass in the current working tree after the fixes in this audit.

Q53. What does the Attack Explanation panel do?

It is a collapsible panel on the incident page. When opened, it loads the explanation for the incident's category and shows the summary, any context such as the source IP for attacks, the likelihood as a coloured badge, OWASP and MITRE tags, and a learn-more link where one exists.

9. Testing, CI and quality

Q54. How many tests are there and what do they cover?

The backend suite passes with 759 tests, 1 skipped and 1 deselected. The tests cover authentication and lockout, RBAC, project isolation, API key security, ingestion, incidents, evidence hashing, the ML bundle gate and tamper detection, the explainer, and the ML review gaps such as the auto-containment blocks. Some integration tests are designed for PostgreSQL but run against SQLite in the default suite, so Postgres-specific behaviour is not fully covered by this run.

Q55. What runs in CI?

The workflow runs ruff for a set of ML and worker files with isort and pyupgrade rules, a ruff format check, the pytest suite with coverage for the ML modules, a slow ML training test, the frontend type-check, lint, tests and build, a bandit scan with medium and high severity as the failure threshold, and a Trivy scan. The dossier previously listed gitleaks and pip-audit as CI steps, but the workflow does not run them, so that claim has been corrected in this document.

Q56. What happened to the test dependencies?

The CI workflow installs only the runtime and dev requirements files, but the test step needs pytest, pytest-asyncio, pytest-cov and pandas. These were missing, so a clean CI runner would have failed. They are now pinned in requirements-dev.txt: pytest 9.0.3, pytest-asyncio 1.4.1, pytest-cov 6.0.0 and pandas 2.2.3. The earlier pin of pytest 8.3.4 had a known advisory, PYSEC-2026-1845, fixed in 9.0.3.

Q57. What is still failing or not gated?

mypy reports 56 errors across 18 files, and it is not a CI gate. The development npm packages still carry 25 advisories, which do not affect the production bundle but should be reviewed. The Terraform configuration has not been validated in this audit because the tool was not available, and gitleaks was not installed, so only a regex scan for common key formats was run.

10. Deployment and operations

Q58. How is the AWS infrastructure defined?

Terraform under terraform/ uses the AWS provider version 5 and defines ECS services, an RDS PostgreSQL database, S3 buckets and SQS queues in modules and environment folders. The Terraform configuration has not been validated in this audit and has never been applied, so it should be described as designed but not deployed.

Q59. How are deployments triggered?

The GitHub Actions deployment workflow runs only on manual dispatch or on version tags that start with v. It does not deploy on every push to main.

Q60. How are secrets handled?

Secrets such as SECRET_KEY and ML_MODEL_SIGNING_KEY are read from environment variables. The root .env file holds them for local use and must never be committed, and the gitignore excludes it. In production they should come from AWS Secrets Manager or ECS task definitions. Terraform state, variable files and override files are also excluded from git.

Q61. What monitoring exists?

The API exposes health and readiness endpoints for liveness checks, and the worker writes a heartbeat file in its container. Structured logging is configured by configure_logging() for both the API and the workers. There is no dashboard or alerting stack configured in the repository yet.

11. Known limitations and open decisions

Q62. What are the main known limitations?

The most important are: the full Compose lifecycle smoke test has not run; the Playwright and login performance tests have not run; the containment path is a placeholder with no EC2 or SSM code, because the legacy worker that held it was removed; the SSE stream and rate limiter are single-replica only; compliance rules have not been reviewed by a lawyer; MFA is not implemented; and the CSP allows inline scripts.

Q63. What open decisions does the owner still need to make?

There are seven: the license, whether to consolidate worker code, where evidence should live (PostgreSQL bytea, S3 WORM, or both), whether to adopt Redis for rate limiting and SSE or document single-replica operation, whether to build TOTP MFA, what the readiness probe should check, and which older audit reports to archive.

Q64. Is there a license?

A placeholder LICENSE file says the code is copyrighted to the two contributors with all rights reserved, and that viewing is for evaluation only. This is a stopgap until a license is chosen. The owner has deferred the decision.

Q65. Why was the legacy worker package removed?

The app/worker package was dead code with broken imports. Its only test imported it, and the live worker is app/workers. Removing it made the codebase reflect what runs. Its containment logic was not carried over, which is why containment is now a placeholder.

Q66. What known documentation drift exists?

The README describes a one-hour JWT expiry and a 100-per-minute rate limit, but the configuration uses 30 minutes and 60 per minute. The README's results section still says the results are synthetic only, even though the shipped model is trained on real CICIDS2017 data. The dossier's CI tool list was also inaccurate. These should be corrected before any public release.

12. Presentation and demo

Q67. What should I say about the ML results in the presentation?

Describe the model as trained on the CICIDS2017 dataset, with 2,830,743 flows, and report the held-out accuracy of 0.9983 and macro F1 of 0.8773, along with the five-fold cross-validation mean of 0.8485. State clearly that the weak classes are the web attacks and Infiltration, that Heartbleed and SQL injection rest on very few flows so their scores are unstable, and that the model selection used a sample and is informative rather than a clean benchmark. Do not claim the results are production-validated.

Q68. How do I demonstrate the platform end to end?

Start Postgres and LocalStack with Docker, run the migrations, start the API on port 8000 and the frontend on port 5173, then log in. Send a high-severity event to /api/v1/events with a project key to create an incident. Open the incident to show the ML verdict, the explanation panel and the evidence vault, then click Verify Hash on an evidence item. The demo should be rehearsed on the target machine first, since the Compose smoke test was not completed in the sandbox.

Q69. What does the presentation deck cover?

The 17-slide deck covers the problem, the architecture, the data, the model comparison, the per-class results, the evaluation method, the project results and the next steps. Its ML slides were updated to use the real CICIDS2017 numbers. The deck validates with the project's validation script.